When the Rules Change: Adaptive Imagination in DreamerV3 Game Agents

Course: CS/CE 351/363 — Artificial Intelligence, Fall 2026
Pathway: Original AI Project
Group: 27
Members: Bilal Ayub (ba09716), Sarosh Mehdi (sy10314), Mustafa Farhan (mq09644)

1. Problem and Motivation

DreamerV3 learns how an environment works and uses predicted future experiences to train a game-playing agent. When game rules change, such as paddle controls or enemy movement, these predictions can become inaccurate and slow learning.

Our question is: Can the agent recover faster by imagining fewer steps when its predictions become less reliable?

We expect shorter imagined sequences to help after a rule change and longer sequences to become useful again as the model adapts. We will test this hypothesis rather than assume it is true.

2. Related Research

1. Hafner et al., Mastering Diverse Domains through World Models (2023; Nature version, 2025). Introduces DreamerV3, which learns a model of its environment and trains an agent using imagined experiences. This is our main reproduction target. Code.
2. Wang et al., COPlanner (2024). Studies how inaccurate predictions can harm learning and how their influence can be limited.
3. Zollicoffer et al., Novelty Detection in Reinforcement Learning with World Models (2025). Studies detecting environmental changes by comparing predictions with actual outcomes.

3. Proposed Method

We will implement a smaller DreamerV3-style agent in PyTorch (we will not reproduce its full benchmarks). It will store past experiences, learn an internal model of game events and rewards, and train a decision-making policy using imagined gameplay. We will build the main learning components ourselves, using the published code as a reference.

Our extension will adjust the length of imagined sequences during training. Prediction error during real gameplay will determine whether the agent imagines 5, 10, or 15 steps. High error will shorten imagination; lower error will allow longer sequences. We will also test whether prediction error is a useful warning signal.

We will first run short tests on these games:

Game	Possible rule change
MinAtar Breakout	Paddle controls or ball movement
MinAtar Space Invaders	Enemy movement or action reliability

Game	Possible rule change
MiniGrid Dynamic Obstacles	Obstacle movement or action noise
Crafter	Movement or resource behavior, if practical

We will select one game for full experiments based on baseline learning, training cost,ability to change rules consistently, and also whether our hardware/compute capability.

4. Original Contribution

We will investigate whether changing imagination length helps a DreamerV3-style agent adjust to new game rules. Uncertainty-aware learning is not new; our contribution is a controlled comparison of this specific method in a changing game environment.

5. Evaluation

We will compare agents using fixed 15-step imagination, fixed 5-step imagination, and adaptive 5/10/15-step imagination. A fixed 10-step version may be added.

Each agent will train under the original rules. We will then change a game mechanic and continue training with matched game-interaction and training-update limits. We will test at least two changes or levels of difficulty and aim for three random seeds per method.

We will measure game score, the performance drop after a change, recovery speed, prediction error, and runtime. The demo will show gameplay, prediction error, and the chosen imagination length.

6. Resources, Timeline, and Backup Plan

Tools: Python, PyTorch, Gymnasium, MinAtar, MiniGrid, Crafter, NumPy, and Matplotlib. Hardware: AMD Radeon RX 9060 XT (16 GB) on Windows. We will test GPU training early and consider Linux with ROCm if needed.

- Week 10: Review papers and test hardware and environments.
- Weeks 11–12: Select one game and build the basic DreamerV3-style agent.
- Week 13: Show initial learning and a game-rule change.
- Week 14: Add adaptive imagination.
- Week 15: Run comparisons and analyze results.
- Week 16: Present the demo and research report.

We will fully evaluate one game. MinAtar Breakout is our preferred option, MiniGrid is the simpler fallback, and Crafter depends on early training results. If needed, we will reduce model size, batch size, or input detail, or switch to a simpler 2D game. We will preserve world-model learning, imagined training, and fair comparisons across methods.